

赛道一：ModelNet40 三维点云分类—— 设计思路（PointMLP）

1. 任务与目标

输入：每个样本 10000 个点，每点 6 维 (x, y, z, n_x, n_y, n_z) 。

输出：40 类形状标签。

评分目标：实例准确率 (iAcc) $\geq 92\%$ ，类平均准确率 (cAcc) $\geq 90\%$ 。

2. 核心思路：为什么选 PointMLP

原方案用 5 个模型（PointNet2 SSG/Msg + DGCNN）集成来凑够分数，但单模型几乎全部不达标。我们希望能用一个模型独立过线，于是调研了近年 SOTA：

方法	会议	OA	特点
PointNet++	NeurIPS'17	91.9%	分层点集抽象，FPS+ball query
DGCNN	TOG'19	92.9%	动态图 CNN，EdgeConv
Point Transformer	ICCV'21	93.7%	向量自注意力
PointMLP	ICLR'22	94.1%	纯 MLP，无卷积/注意力
PointNeXt	NeurIPS'22	94.1%	PointNet++改进训练
Point-MAE	ECCV'22	93.8%	掩码预训练（需额外数据）

选择 PointMLP 的理由：

1. 纯 MLP 架构，不依赖卷积或注意力，计算高效；
2. 官方 OA 94.1% / mAcc 91.5%，单模型即可过线；
3. 有完整开源代码，可直接复现；
4. xyz-only 输入（无需法线），减少了跨分布的泛化风险。

3. PointMLP 架构详解

PointMLP 的核心洞察是：点云分类的关键不是复杂的算子，而是合适的局部几何建模。架构由三部分组成：

3.1 LocalGrouper（局部聚合）

- FPS（最远点采样）在空间中均匀降采样中心点；
- KNN 在每个中心点周围取 $k = 24$ 个最近邻；
- **Geometric Affine 归一化**：以锚点（中心点）为参考，将邻域内各点标准化： $\mathbf{x}' = (\mathbf{x} - \mathbf{x}_{\text{anchor}}) / \sigma$ ，再通过可学习参数 α, β 做仿射变换： $\mathbf{y} = \alpha \cdot \mathbf{x}' + \beta$ ；
- 最后拼接锚点特征作为残差上下文（类似残差连接的隐式形式）。

3.2 Pre & Post Extraction（前后残差 MLP）

- **PreExtraction**：在邻域的 k 个点上做共享 MLP + max-pool，升维到目标通道数，再加若干个残差 block；
- **PosExtraction**：在池化后的全局特征上做通道间残差 MLP。
- 双层残差设计（邻域内 + 通道间）保证深层梯度流动。

3.3 整体结构

4 级层级结构，通道逐级翻倍（ $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$ ）、点数逐级减半（ $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64$ ）：

Stage	输入点数	KNN k	输入通道	输出通道	残差 block 数
Embedding	1024	—	3	64	1 (Conv1d)
Stage 1	512	24	64	128	2+2
Stage 2	256	24	128	256	2+2
Stage 3	128	24	256	512	2+2
Stage 4	64	24	512	1024	2+2
Classifier	—	—	1024	40	FC→512→256→40

全局 max-pool 聚合后将 1024 维特征送入三层 FC 分类头，中间带 BN+ReLU+Dropout(0.5)。总参数量 **13.27M**。

4. 训练过程

4.1 数据预处理 (preprocess.py)

将教师提供的 ~9843 个 .txt 文件 (每文件 10000 行×6 列) 读入并统一为 data.npy (~1.2GB, fp16), 同时生成 labels.npy 和 shape_names.npy。后续通过 mmap 读取, 避免大量小文件 I/O。

4.2 分层验证划分

stratified_split(labels, val_ratio=0.1, seed=2026) 对每类独立做 9:1 分层采样, 得到固定的 8877 训练 / 966 验证样本。所有超参调优仅看 val, test 最后一次运行。

4.3 训练配方 (PointMLP 官方配方)

- 优化器: SGD momentum=0.9, weight_decay= 2×10^{-4} (不用 Adam/AdamW; PointMLP 论文指出 SGD 是该架构的最佳搭档);
- 学习率: Cosine annealing, 从 0.1 退火到 0.005;
- 损失函数: CrossEntropyLoss(label_smoothing=0.2);
- 混合精度: torch.cuda.amp + GradScaler, 训练加速约 30%;
- Batch size: 32;
- 训练轮数: 200 epochs, 每 2 个 epoch 在 val 上评测一次;
- 增强 (仅 xyz):
 - 各向异性缩放 $[0.66, 1.5]^3$
 - 平移 ± 0.2
 - 不做旋转 (xyz-only 模型旋转增强反而有害)

训练在单张 RTX A6000 (49GB) 上完成, 约 97 秒/epoch, 总计 ~5.4 小时。

4.4 Checkpoint 策略

每轮保存三个 ckpt:

- pointmlp.pt —— 至今 val iAcc 最高者;
- pointmlp.bestcacc.pt —— 至今 val cAcc 最高者;
- pointmlp.pt.last —— 最后一轮。

5. TTA 推理 (predict.py)

- 对每个样本, 先在整个 10000 点上做 center + unit-ball 归一化;
- 随机子采样 1024 个点, 过模型取 softmax, 重复 10 次;
- 10 轮 softmax 求平均后 argmax 得最终预测。

支持三种输入模式: npy 文件、txt 目录、id 列表+根目录。另设 --fast 应急模式 (TTA=1, CPU 可跑 ~30s)。

6. 实验结果

6.1 验证集 (966 样本, seed=2026 val split)

Checkpoint	Epoch	val iAcc	val cAcc
best iAcc	134	93.48%	90.00%
best cAcc	182	93.37%	90.61%
last	200	92.96%	88.78%

6.2 测试集 (2468 样本, 教师下发)

采用 best_cAcc checkpoint 进行最终评测:

方案	test iAcc	test cAcc	是否达标
旧方案（5 模型集成 + TTA）	92.38%	89.39%	×
PointMLP（本方案）	93.23%	90.34%	✓

双指标均超过 92% / 90% 门槛。单 PointMLP 模型在 iAcc 上比旧 5 模型集成提升 0.81%，cAcc 提升 0.95%。

6.3 主要困难类别

flower_pot 在所有方案中持续表现最差（test 上仅 4/20 recall），系花瓶/花盆/盆栽三类几何形状高度相似导致的固有分类歧义。ICML 2023 论文 “ModelNet40: What is left?” 也指出该问题属于标签噪声和类间固有歧义，非模型能力问题。

7. 对比旧方案

维度	旧方案（5 模型集成）	新方案（PointMLP）
模型数量	5 个（3 类架构）	1 个
总参数量	~8.2M × 5	13.27M
训练时间	~7-8 h × 5	~5.4 h
推理时间 (TTA=10)	~10-20 min	~5-10 min
单模 iAcc/cAcc	全部 < 92%/< 90%	93.19%/90.34%
部署复杂度	需同时加载 5 个模型	单文件 ckpt

8. 结论

PointMLP 以更简单的架构（纯 MLP，无卷积/注意力）、更少的模型数量（1 vs 5）、更短的训练时间，在 ModelNet40 上实现了超越原集成方案的结果，且无需法线信息（仅 xyz 输入），降低了跨预处理流水线的分布偏移风险。